



Python Course 2025/26/2

Lecture 3: Flow, Strings, and Lists

Course coordinator: Dr Máté Tejfel

Lecturer: Zoltán Kégli



Lecture Overview

Topics

- Logic operators and comparison
- Flow control with `if / elif / else`
- Loops with `while` and `for`
- Strings: indexing, slicing, immutability
- Lists: operations and practical usage

Session Format

Concepts

Examples

Live Coding

We will build from core language building blocks to a mini command-based To-do app.

Tip: Use horizontal arrows for sections and vertical arrows for deep dives.



Logic and Comparison

Boolean thinking for decisions in code



Logic Operators on Booleans

A and B are Boolean values.

- not A: True if A is False
- A and B: True only if both are True
- A or B: True if at least one is True

Operator precedence: not > and > or.

Truth Table

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False



Truthy, Falsy, and Short-Circuiting

Falsy values (common)

`False`, `0`, `0.0`, `""`, `[]`, `None`

Everything else is usually truthy.

```
items = []
if items:
    print("has items")
else:
    print("empty")
```

```
name = ""
```

```
# short-circuit: second part is skipped
if name and name[0] == "A":
    print("Starts with A")
```

Fun Python fact

`and` `and` `or` return one of the original operands.

```
0 or 42 # 42
"hello" and "world" # "world"
```



Comparison Operators

Comparison expressions evaluate to `True` or `False`.

Operator	Meaning	Example	Result
<code>=</code>	Equal to	<code>5 = 5</code>	True
<code>≠</code>	Not equal to	<code>True ≠ False</code>	True
<code>></code>	Greater than	<code>7 > 10</code>	False
<code><</code>	Less than	<code>"Ann" < "Bob"</code>	True
<code>≥</code>	Greater or equal	<code>5.0 ≥ 5.0</code>	True
<code>≤</code>	Less or equal	<code>4 ≤ 2</code>	False
<code>in</code>	Membership test	<code>"py" in "python"</code>	True
<code>is</code>	Identity (same object)	<code>x is None</code>	True/False

Strings compare lexicographically. Cross-type equality usually returns `False` (for example: `'3' = 3`).
Use `is` for identity checks (especially `is None`), and `=` for value equality. Chained comparisons are valid: `0 < x < 10` .



Checkpoint: Logic and Comparison

Q1

`not not not True` = ?

A) True

B) False

C) SyntaxError

D) Existential crisis

Show Answer

Q2

`True + True` = ?

A) True

B) 2

C) TypeError

D) TrueTrue

Show Answer

Q3

`"banana" > "apple"` = ?

A) True

B) False

C) TypeError

D) Depends on ripeness

Show Answer



Flow Control

From straight-line code to decisions and repetition

Why Flow Control?

Straight-line execution

- Runs top to bottom in fixed order
- Always follows the same path
- Limited for real-world problem solving

```
x = 5  
y = x + 2  
print(y) # always prints 7
```

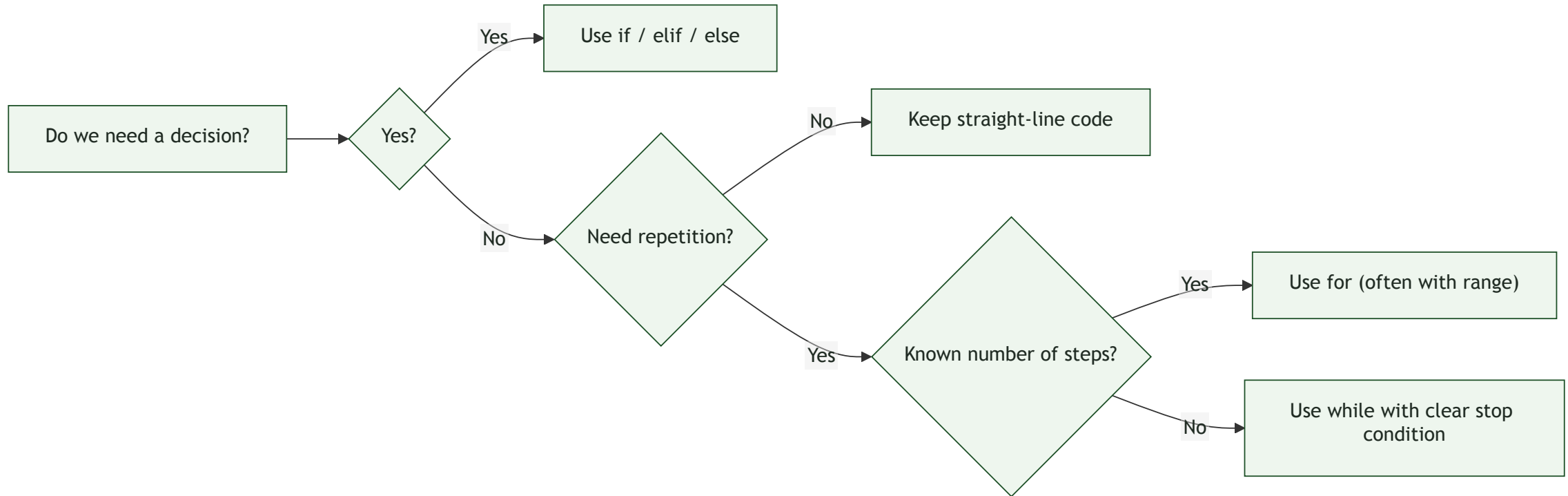
Real programs need:

Decisions

Repetition



Flow Control Decision Guide



Quick rule: choose `for` when you iterate over known data, and `while` when logic decides when to stop.

Branching with if

Use `if` to execute a block only when a condition is true.

```
x = 10

if x > 5:
    print("x is large")
```

Pattern

```
if <condition>:
```

Indented block executes when condition is True.



If...else

Add `else` to define the "otherwise" path.

```
age = 16

if age ≥ 18:
    print("Adult")
else:
    print("Minor")
```



If...elif...else

Handle multiple mutually exclusive conditions.

```
score = 85

if score ≥ 90:
    grade = "A"
elif score ≥ 75:
    grade = "B"
else:
    grade = "C"
```

`elif` is evaluated only if the previous conditions were False.



Importance of Indentation

```
age = 16

if age ≥ 18:
    print("Adult")
    student = False
else:
    print("Minor")
```

Why it matters

- Defines code blocks in Python
- Keeps conditional logic readable
- Incorrect indentation causes syntax errors
- Use 4 spaces (PEP 8); avoid mixing tabs/spaces

Indentation vs Scoping



Java: braces create block scope

```
int age = 20;

if (age ≥ 18) {
    String status = "adult";
}

System.out.println(status); // compile-time error
```

Variables declared inside `{ }` are not visible outside that block.

Python: indentation defines blocks

```
age = 20

if age ≥ 18:
    status = "adult"

print(status) # "adult"
```

Indentation groups statements, but `if` does not create a new scope.

In Python, `if`, `for`, and `while` blocks do not create new scopes. New scopes are introduced by functions, classes, and modules.

Checkpoint: Flow Control



Q4

How many times does this print "hi" ?

```
while False:  
    print("hi")
```

- A) 0
- B) 1
- C) Infinite
- D) Depends on mood

Show Answer

Q5

What happens if you indent with 3 spaces?

- A) SyntaxError
- B) It works
- C) Always IndentationError
- D) TabError

Show Answer



Loops

Repeat work cleanly and safely

Loops in Python



Main loop types

- `while`: repeat while condition is true
- `for`: iterate over sequence/iterable

Why loops?

- Avoid copy-paste repetition
- Process data item by item
- Retry/search/countdown patterns



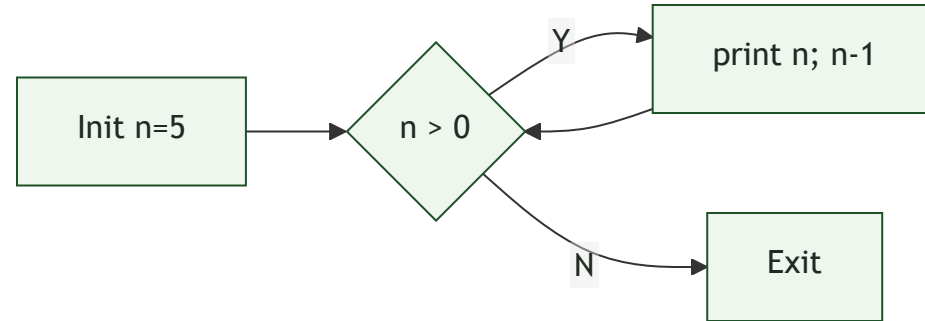
While Loop

`while` runs its block as long as the condition remains true.

```
n = 5
while n > 0:
    print(n)
    n -= 1
```

Always make progress toward ending the condition.

While Loop Flow



The condition is rechecked after each iteration until it becomes false.



Infinite Loop

```
while True:  
    print("I'm stuck in an infinite loop")
```

```
# Stop with Ctrl + C
```

Useful only when you include a clear exit strategy (break, event, signal).



For Loop

`for` iterates over iterable values such as strings, lists, and tuples.

```
for letter in "Python":  
    print(letter)  
print("Done")
```

While and For with Numbers



While version

```
n = 0
while n < 5:
    print(n)
    n += 1
```

For version

```
for n in range(5):
    print(n)
```

Use `for` when iteration count is known.

Range, Break, and Continue

range()

range(start, stop, step)

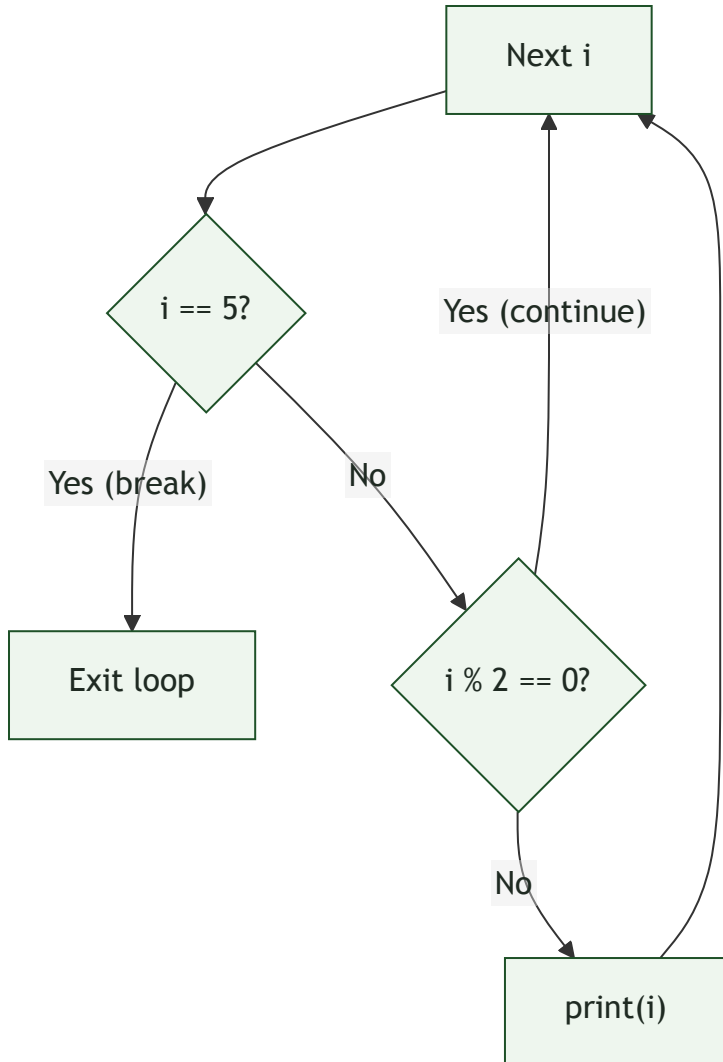
```
print(list(range(5)))      # 0..4
print(list(range(2, 8)))   # 2..7
print(list(range(0, 20, 3))) # 0,3,6, ...
```

Loop control

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
```

for ... else exists too: else runs only if no break happened.

Break and Continue



```
for i in range(10):  
    if i == 5:  
        break  
    if i % 2 == 0:  
        continue  
    print(i)
```

Output: 1, 3

Loop Variable Scoping

Java: loop var stays in loop scope

```
for (int i = 0; i < 3; i++) {  
    System.out.println(i);  
}  
System.out.println(i); // compile-time error
```

`i` is declared in the `for` header and is not visible after the loop.

Python: loop var remains accessible

```
for i in range(3):  
    print(i)  
  
print(i) # 2
```

`i` stays in the enclosing scope and keeps the last iterated value.

Python loop variables behave like other names in the surrounding scope. This is why `print(i)` after the loop works in Python.



Flow Control: Summary

Branching

`if / elif / else` picks one path based on conditions.

Repetition

`while` for condition-driven loops,
`for` for sequence-driven loops.

Control

`range()`, `break`, and `continue` fine-tune behavior.

Core idea:

Flow control defines how your program thinks, decides, and repeats.

Checkpoint: Loops



Q6

What does `range(10, 0)` produce?

A) 10..1

B) Empty

C) Error

D) [10, 0]

Show Answer

Q7

What is printed?

```
for i in range(3):  
    pass  
print(i)
```

A) 0

B) 2

C) 3

D) NameError

Show Answer



Strings

Immutable sequences of characters

Strings in Python

Core properties

- Sequence of case-sensitive characters
- Supports comparison operators
- Supports membership checks via `in`
- Length available via `len()`

```
s = "abc"  
print(len(s)) # 3
```

String Indexing

Use square brackets to access characters at positions.

```
s = "abc"

print(s[0])    # "a"
print(s[1])    # "b"
print(s[2])    # "c"
print(s[-1])   # "c"
print(s[-2])   # "b"
```

Accessing out-of-range indices raises an error.



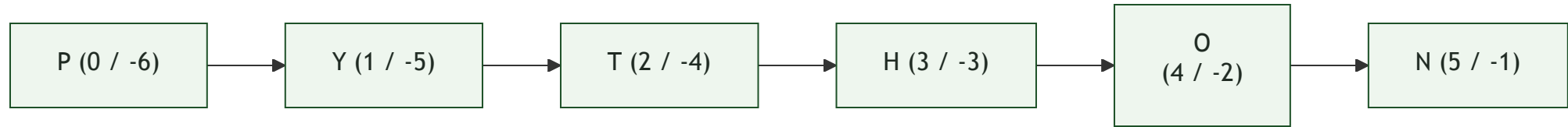
String Slicing

Slicing syntax: `[start:stop:step]`

```
s = "abcdefgh"

print(s[3:6])      # "def"
print(s[3:6:2])    # "df"
print(s[::])       # "abcdefgh"
print(s[::-1])     # "hgfedcba"
print(s[4:1:-2])   # "ec"
```


String Indexing Map



Read it left to right

`s[0]` is first char, `s[-1]` is last char.

Slice examples

`s[1:4]` - from index 1 up to 4 (excluded)

`s[::-1]` - full reverse

String Immutability



```
s = "hello"  
s[0] = "y"      # TypeError
```

```
s = "hello"  
s = "y" + s[1:]  # "yello"
```

Strings cannot be modified in-place. You create a new string instead.



Common String Methods

- `.lower()`, `.upper()`
- `.strip()` remove edge whitespace
- `.split()` and `"sep".join( ... )`
- `.replace(old, new)`
- `.find()`, `.count()`

```
command = "  Add  "
clean = command.strip().lower() # "add"

words = "python is fun".split()
slug = "-".join(words)
print(slug.replace("fun", "great"))
```

String methods return a new string; they do not mutate the original.

f-Strings (Recommended Formatting)

```
name = "Ada"
score = 97.5
print(f"{name} scored {score:.1f}%")

for i, task in enumerate(tasks, start=1):
    print(f"{i}. {task}")
```

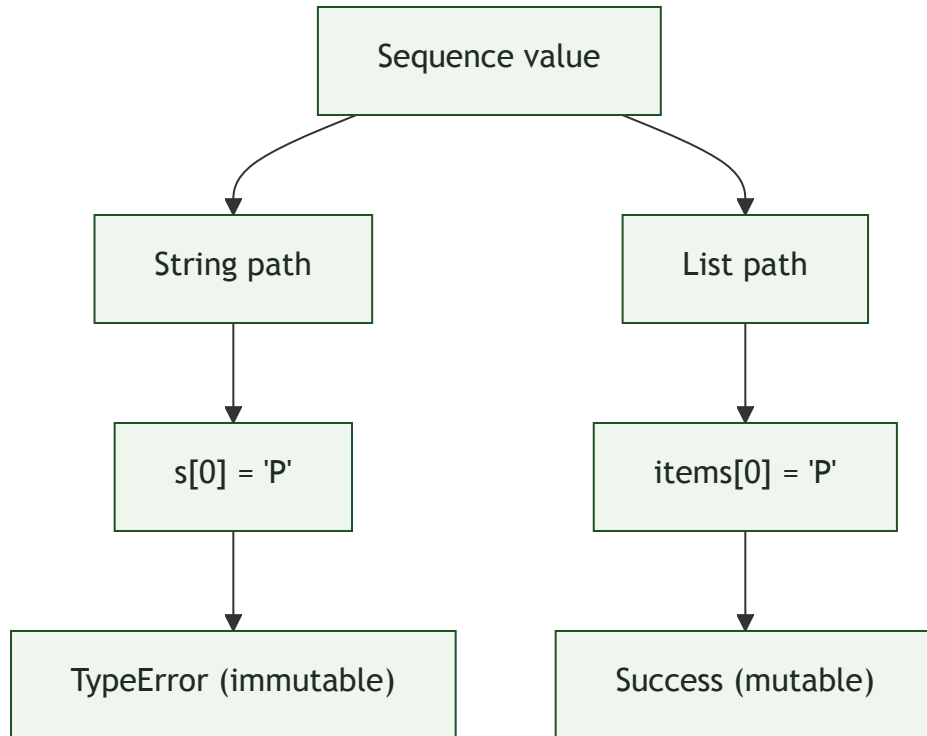
Prefix with `f`, then put expressions inside `{}`.

Why use f-strings?

- Most readable string formatting style
- Fast and explicit for mixed text + values
- Supports formatting options like `:.2f`

```
pi = 3.14159
print(f"{pi:.2f}") # 3.14
```

Mutable vs Immutable



```
s = "python"
# s[0] = "P" # TypeError

items = ["python", "java"]
items[0] = "Python" # works
```

Rule of thumb: if you need frequent edits, lists are the better container.

Checkpoint: Strings



Q8

`"hello"[::-1]` = ?

A) hello

B) olleh

C) h

D) Error

Show Answer

Q9

`"ha" * 3` = ?

A) ha3

B) hahaha

C) Error

D) hhhaaa

Show Answer

Q10

Can we do `s[0] = "X"` on a string?

A) Yes

B) No

C) Only Tuesdays

D) With admin rights

Show Answer



Lists

Mutable collections for real-world data

Lists in Python

Why lists?

- Store collections in one variable
- Can contain mixed data types
- Mutable: editable after creation

```
my_list = [1, 2, 3, 4, 5]
fruits = ["apple", "banana", "cherry"]
mixed = [1, "apple", True]
empty = []
```




Accessing List Elements

- Indexing: `list[index]`
- Negative indexing: `list[-1]` for last item
- Slicing: `list[start:stop:step]`
- Membership: `value in list`

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])      # "apple"
print(fruits[-1])     # "cherry"
print(fruits[1:])     # ["banana", "cherry"]
print(fruits[::-1])   # reversed
print("apple" in fruits) # True
```

Modifying List Elements



Add or replace

```
tasks = ["study", "exercise"]  
tasks[0] = "review"  
tasks.append("sleep")  
tasks.insert(1, "code")
```

Remove

```
tasks.remove("exercise")  
del tasks[0]  
done = tasks.pop() # removes + returns last item
```



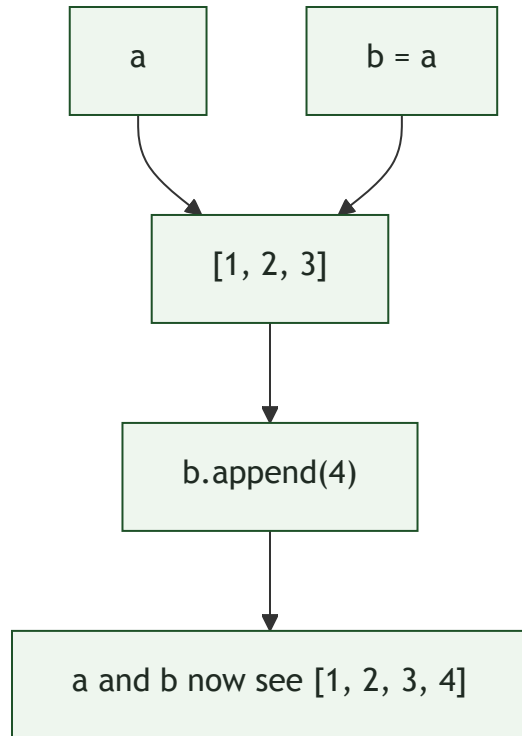
Useful List Functions

- `len(list)`: length
- `sorted(list)`: new sorted list
- `.sort()`: sort in place (returns `None`)
- `.reverse()`: reverse in place
- `.extend(items)` vs `.append(item)`

```
nums = [3, 1, 2]
print(sorted(nums)) # [1, 2, 3]
print(nums)         # [3, 1, 2]
nums.sort()         # nums changed in place

a = [1, 2]
a.append([3, 4])     # [1, 2, [3, 4]]
a.extend([5, 6])     # [1, 2, [3, 4], 5, 6]
```

List Aliasing Trap



```
a = [1, 2, 3]
b = a
b.append(4)
print(a)  # [1, 2, 3, 4]

# safe copy
c = a.copy()  # or a[:]
```

Assignment shares the same list object. Copy when you need independence.

Iterating over Lists with for

Direct iteration

Use `for item in my_list` when you only need each value.

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Index-based iteration

Use `range(len( ... ))` when you need positions.

```
for i in range(len(fruits)):
    print(i, fruits[i])
```

enumerate(): Index + Value

Recommended pattern

```
fruits = ["apple", "banana", "cherry"]  
  
for idx, fruit in enumerate(fruits):  
    print(idx, fruit)
```

enumerate() returns pairs: (index, value).

Why use enumerate?

- Cleaner than `range(len( ... ))`
- Less error-prone index handling
- Easy to start counting from another value

```
for n, fruit in enumerate(fruits, start=1):  
    print(n, fruit)
```

List Operators

Concatenation

+ combines lists

Repetition

* repeats a list

```
fruits = ["apple", "banana"]
berries = ["blueberry", "raspberry"]

print(fruits + berries)
print(fruits * 3)
```





Why Lists Matter

Collect Data

Store many related values together.

Enable Loops

Iterate through user input and file data.

Foundation

Essential for data processing and analysis.



Lists: Summary

Data

- Mutable, ordered collections
- Can store mixed data types
- Use indexing and slicing to access values

Operations

- Add/update: `append()`, `insert()`, indexing
- Remove: `remove()`, `del`, `pop()`
- Reorder/compose: `sorted()`, `sort()`, `reverse()`, `+`, `*`

Iteration

- `for item in items` for values
- `range(len(items))` when index is needed
- `enumerate(items)` for index + value pairs
- Copy when needed: `items.copy()`

Lists are central for collecting, transforming, and iterating over program data.

Checkpoint: Lists



Q11

`[1, 2, 3] * 0` returns?

A) `[0, 0, 0]`

B) `[]`

C) Error

D) `[1, 2, 3]`

Show Answer

Q12

What is printed?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

A) `[1, 2, 3]`

B) `[1, 2, 3, 4]`

C) Error

D) `[4]`

Show Answer



To-do App Demo

Putting flow control, strings, and lists together



Iteration 4

Feature 4.1

Manage **3 hardcoded tasks**. Store and print their descriptions using lists and loops.

Feature 4.2

Ask the user for one extra task, append it to the list, then print all tasks with numbering.



Iteration 5

Implement a main command loop with this interface:

```
while True:
    command = input("Enter command (add/view/exit): ").strip().lower()

    if command == "add":
        ...
    elif command == "view":
        ...
    elif command == "exit":
        break
```

add

view

exit



Lecture Summary

Flow Control

- Use `if / elif / else` for branching
- Use `while` and `for` for repetition
- Use `range()`, `break`, `continue` to guide loops

Strings and Lists

- Strings are immutable but sliceable/indexable
- Lists are mutable and ideal for collections
- Together they power practical mini apps

Bonus: Bitwise Operators

Operator Cheat Sheet

Operator	Meaning
&	bitwise AND
	bitwise OR
^	bitwise XOR
~	bitwise NOT
<<	left shift
>>	right shift

Tip: do not confuse bitwise operators with logical `and` / `or` / `not`.

Simple Examples

```
5 # 0101
3 # 0011

5 & 3 # 0001 = 1
5 | 3 # 0111 = 7
5 ^ 3 # 0110 = 6
1 << 3 # 1000 = 8
8 >> 1 # 0100 = 4
```

Where bitwise operators are useful

- Flags and masks (permissions, status bits)
- Binary and networking tasks
- Performance-minded low-level operations



Thank You for Your Attention!

"Talk is cheap. Show me the code."

-- Linux Torvalds

if/elif/else

loops

strings

lists